NeuroNudge: Early Neurodivergence Screening Platform

Project Report v1.0 · Local web app (localhost) · Frontend: Lovable (React) · Backend: Python/FastAPI · AI: OpenCV + MediaPipe + Gemini API

1. Overview

NeuroNudge is a locally hosted web application that runs a short, staged, game-like assessment on a child or adult and produces an AI-written **screening report** about indicators of ADHD, dyslexia and autism.

Three stages, one per condition:

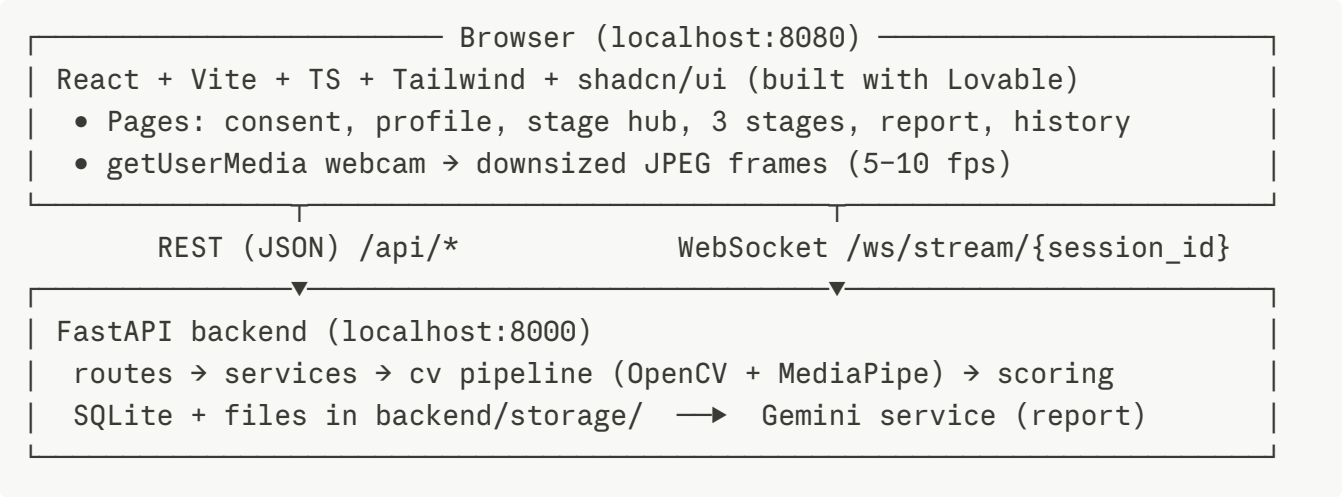
1. **Stage 1: ADHD.** Stimulating images and videos play while the webcam tracks gaze, blinks, head movement and facial engagement.
2. **Stage 2: Dyslexia.** Multiple-choice questions on letters, sounds, words and rhymes, with response times recorded.
3. **Stage 3: Autism.** Social-situation questions plus webcam analysis of eye contact and repetitive movement.

All raw and derived data is stored on the laptop. At the end, the backend computes deterministic scores, and Gemini turns those scores into a readable report: observed symptoms, a ranked list of most to least likely condition (if any), and suggested next steps.

2. Scope and Safety Statement (must be shown in the UI)

This tool is a **screening aid, not a diagnosis**. It must never say "the user has autism/ADHD/dyslexia". It reports *indicator levels* (Low / Moderate / High) and recommends a qualified professional for confirmation. Webcam-based gaze and expression analysis is approximate; results depend on lighting, camera quality and the user's age and attention. Every report carries a disclaimer, and users give explicit consent before the camera starts.

3. System Architecture



Why frames go over a WebSocket: the stimulus is on the browser screen, so the browser owns the webcam and timestamps every frame against what was on screen. The backend runs the OpenCV analysis. (Alternative if you want less load: run MediaPipe in the browser and send only landmark numbers.)

4. Tech Stack

Layer	Choice
Frontend	React 18, Vite, TypeScript, Tailwind CSS, shadcn/ui, React Router, TanStack Query, Framer Motion, Recharts
Backend	Python 3.11, FastAPI, Uvicorn, Pydantic v2, SQLAlchemy
Computer vision	OpenCV (`opencv-python`), MediaPipe Face Mesh with iris landmarks, MediaPipe Hands (optional), NumPy, SciPy
AI report	Google Gemini API via `google-genai` SDK, structured JSON output
Storage	SQLite file + per-session JSON files, all inside `backend/storage/`
Tooling	VS Code, `concurrently` (run both servers), Git

5. User Flow

1. **Landing page** with purpose, disclaimer and "Start".
2. **Consent** (guardian consent if age < 18), camera permission, data stays local.
3. **Profile**: nickname, age, language, who is taking the test (child / adult).
4. **Camera check**: face visible, lighting OK, calibration (look at 5 dots).
5. **Stage hub**: three cards showing progress; stages run in order but can be resumed.
6. **Stage 1 → Stage 2 → Stage 3.**
7. **Processing screen** while scoring and Gemini run.
8. **Report page** with indicator chart, symptoms, ranking, recommendations, disclaimer, PDF/print.
9. **History page** listing earlier local sessions, with delete-all-data button.

6. Feature Specification

6.1 Stage 1: ADHD (attention and activity)

Task (about 3 minutes): 6–8 short stimulus blocks of 20–30 s each: bright animations, a calm slow scene, a fast-cut video, a "find the moving object" mini-game, and a sustained-attention task (press when a star appears).

CV signals captured per block:

- Gaze-on-screen ratio and gaze dispersion (how widely the eyes wander)
- Number and length of look-aways
- Fixation duration and saccade rate
- Blink rate
- Head movement amount (pose variance) and fidgeting
- Facial engagement proxy (brow and mouth movement)

Behaviour signals: reaction time, missed targets, false presses (impulsivity).

6.2 Stage 2: Dyslexia (reading and phonology)

Task (about 4 minutes, 15–20 MCQs, no camera needed):

- Letter confusion (b/d, p/q, m/w)
- Rhyme and sound matching, with text-to-speech audio

- Real word vs. non-word choice
- Spelling choice (pick the correct spelling)
- Sequence tasks (days, alphabet order)
- Short sentence comprehension

Signals: accuracy per category, response time per question, hesitation (hover or change-of-answer count), reversal-error count. Question banks are age-banded (5–7, 8–12, 13+) and stored as JSON on the backend.

6.3 Stage 3: Autism (social communication and repetitive behaviour)

Part A, questions (about 3 minutes): 10–12 social-situation MCQs with illustrated scenes (e.g. a friend looks sad, someone says a joke sarcastically, a group invites you to play). Categories: emotion recognition, perspective-taking, figurative language, social rules.

Part B, camera (about 2 minutes): a friendly animated face talks to the user and a neutral waiting screen follows.

CV signals: eye-contact ratio toward the face region of the stimulus, gaze aversion, repetitive head or body movement (detected by periodicity analysis, i.e. FFT of head and hand landmark motion), limited facial expression range.

6.4 Final Report

Contents: participant summary; per-stage metrics; indicator level for each of ADHD, dyslexia and autism with a confidence note; observed symptoms in plain language; **most likely and least likely** condition (or "no notable indicators"); recommended next steps; disclaimer. Exportable as PDF (browser print stylesheet).

7. Folder and File Structure

```
neuronudge/
├── README.md
├── .gitignore                    # ignores .env, node_modules,
backend/storage/
├── package.json                  # root scripts: dev runs frontend + backend
together
|
├── frontend/                     # built with Lovable, then cloned to VS
Code
|   ├── package.json
```

```
|   ├── vite.config.ts            # dev proxy: /api and /ws → http://localhost:8000
|   ├── tailwind.config.ts
|   ├── .env.example              # VITE_API_BASE_URL=http://localhost:8000
|   ├── public/stimuli/           # images, videos, audio for stages
|   └── src/
|       ├── main.tsx
|       ├── App.tsx               # router + providers
|       ├── pages/
|       |   ├── Landing.tsx
|       |   ├── Consent.tsx
|       |   ├── Profile.tsx
|       |   ├── CameraCheck.tsx
|       |   ├── StageHub.tsx
|       |   ├── AdhdStage.tsx
|       |   ├── DyslexiaStage.tsx
|       |   ├── AutismStage.tsx
|       |   ├── Processing.tsx
|       |   ├── Report.tsx
|       |   ├── History.tsx
|       |   └── NotFound.tsx
|       ├── components/
|       |   ├── layout/           # Header, Footer, ProgressStepper
|       |   ├── camera/           # WebcamPreview, CalibrationDots, FaceStatusBadge
|       |   ├── stimuli/          # StimulusPlayer, TargetGame
|       |   ├── questions/        # McqCard, AudioPrompt, SceneCard
|       |   ├── report/           # IndicatorChart, SymptomList, RankingCard
|       |   └── ui/               # shadcn components
|       ├── hooks/
|       |   ├── useWebcam.ts      # getUserMedia, frame capture
|       |   ├── useFrameStream.ts # WebSocket frame sender
|       |   ├── useSession.ts     # session id + stage progress
|       |   └── useStageTimer.ts
|       ├── lib/
|       |   ├── api.ts            # typed fetch wrappers for every endpoint
|       |   ├── types.ts          # TS types mirroring backend schemas
|       |   └── constants.ts
|       └── store/sessionStore.ts # Zustand or Context
|
└── backend/                      # built by teammates
    ├── requirements.txt          # fastapi, uvicorn[standard], opencv-python, mediapipe,
```

```
    |                               # numpy, scipy, sqlalchemy, pydantic,
python-dotenv,
    |                               # google-genai, websockets, python-
multipart
    ├── .env.example                # GEMINI_API_KEY=, GEMINI_MODEL=,
FRONTEND_ORIGIN=
    ├── run.py                      # uvicorn launcher
    └── app/
        ├── main.py                 # FastAPI app, CORS, router registration
        ├── config.py               # settings from .env
        ├── api/
        |   ├── health.py           # GET /api/health
        |   ├── sessions.py         # create, get, list, delete sessions
        |   ├── stages.py           # stage start/finish, question fetch,
answer submit
        |   ├── reports.py          # generate and fetch report
        |   └── ws_stream.py        # WebSocket for webcam frames
        ├── cv/
        |   ├── frame_decoder.py    # base64/JPEG → OpenCV image
        |   ├── face_mesh.py        # MediaPipe landmarks wrapper
        |   ├── gaze.py             # iris-based gaze + calibration mapping
        |   ├── blink.py            # eye aspect ratio blink detection
        |   ├── head_pose.py        # pitch/yaw/roll via solvePnP
        |   ├── expression.py       # brow/mouth geometry features
        |   ├── repetitive_motion.py  # FFT periodicity on landmark signals
        |   └── aggregator.py       # per-block metric summaries
        ├── scoring/
        |   ├── adhd_scoring.py
        |   ├── dyslexia_scoring.py
        |   ├── autism_scoring.py
        |   └── combine.py          # indicator levels + ranking
        ├── services/
        |   ├── gemini_service.py # prompt build, call, JSON validation,
retry
        |   ├── report_builder.py # assembles final report object
        |   └── session_service.py
        ├── db/
        |   ├── database.py         # SQLite engine
        |   ├── models.py           # tables (section 8)
        |   └── crud.py
        ├── schemas/                # Pydantic models: session, stage, metrics,
report
        ├── data/
```

```
        |   ├── questions/dyslexia_5_7.json, dyslexia_8_12.json,
dyslexia_13plus.json
        |   ├── questions/autism_social.json
        |   └── prompts/report_system_prompt.txt
        └── storage/              # created at runtime, git-ignored
            ├── neuronudge.db
            └── sessions/<session_id>/{metrics.json, report.json}
```

8. Data Model (SQLite)

Table	Key columns
`sessions`	id, nickname, age, language, consent_at, status, created_at
`stage_runs`	id, session_id, stage (`adhd` / `dyslexia` / `autism`), started_at, finished_at, status
`cv_blocks`	id, stage_run_id, block_label, start_ms, end_ms, metrics_json
`answers`	id, stage_run_id, question_id, category, chosen_option, correct, response_ms
`scores`	id, session_id, stage, features_json, indicator_score (0–100), level
`reports`	id, session_id, report_json, model_name, created_at

Raw video is never saved. Frames are analysed in memory and discarded; only numeric metrics are stored.

9. API Contract

Method	Path	Purpose
GET	`/api/health`	Backend alive check
POST	`/api/sessions`	Create session from profile + consent, returns `session_id`

Method	Path	Purpose
GET	`/api/sessions`	List past sessions
GET	`/api/sessions/{id}`	Session with stage progress
DELETE	`/api/sessions/{id}`	Delete one session's data
DELETE	`/api/sessions`	Delete all local data
POST	`/api/sessions/{id}/stages/{stage}/start`	Begin a stage
POST	`/api/sessions/{id}/stages/{stage}/finish`	End stage, returns stage summary
GET	`/api/questions/{stage}?age=`	Fetch MCQs for dyslexia or autism
POST	`/api/sessions/{id}/answers`	Submit an answer with response time
POST	`/api/sessions/{id}/block-marker`	Tell backend which stimulus block is on screen
POST	`/api/sessions/{id}/report`	Run scoring + Gemini, returns report
GET	`/api/sessions/{id}/report`	Fetch stored report
WS	`/ws/stream/{id}?stage=`	Send frames, receive live face/gaze status

WebSocket protocol: client sends `{ "t": <ms>, "block": "adhd_video_2", "frame": "<base64 jpeg 320x240>" }`. Server replies about once per second with `{ "face_found": true, "lighting": "ok", "gaze_on_screen": 0.82 }` so the UI can warn if the face is lost. Calibration frames use `block: "calibration_k"`.

10. Computer Vision Pipeline

1. Decode JPEG → OpenCV image, resize to 320×240, convert to RGB.
2. MediaPipe Face Mesh (`refine_landmarks=True`) returns 468 face points plus iris points.
3. **Gaze:** iris centre relative to eye corners gives a horizontal and vertical ratio; a 5-point calibration maps ratios to screen regions. Output: on-screen flag, region, dispersion.

4. **Blink:** eye aspect ratio below a threshold for 2+ frames counts as a blink.
5. **Head pose:** `cv2.solvePnP` on six landmarks gives yaw, pitch and roll; variance is the movement score.
6. **Expression:** mouth-open ratio, brow-raise distance and smile width give a coarse engagement and expressiveness index (explicitly not emotion diagnosis).
7. **Repetitive motion:** keep a 10–15 s rolling window of head (and optional hand) landmark positions; FFT finds a dominant frequency between 0.5 and 4 Hz with high power, flagged as a repetitive pattern.
8. **Aggregation:** per block, compute means, variances, counts and ratios and write one `metrics_json` row. Frames that miss a face are counted as `face_lost_ratio`.

11. Scoring and Gemini Report

Step 1, deterministic scoring (backend code, not AI). Each stage converts its features into a 0–100 indicator score against age-adjusted thresholds kept in config files so the team can tune them. `combine.py` maps scores to Low (<35), Moderate (35–65) or High (>65) and ranks the three conditions.

Step 2, Gemini writes the narrative. The backend sends Gemini a compact JSON of participant age, per-stage metrics, scores, levels and ranking. Gemini is told to explain, not re-score, never to diagnose, and to return JSON matching this shape:

```
{
  "summary": "string",
  "indicators": [
    {"condition": "adhd|dyslexia|autism", "level": "low|moderate|high",
     "observed_symptoms": ["string"], "evidence": ["metric-based sentence"]}
  ],
  "most_likely": "adhd|dyslexia|autism|none",
  "least_likely": "adhd|dyslexia|autism",
  "recommendations": ["string"],
  "confidence_note": "string",
  "disclaimer": "string"
}
```

The backend validates the JSON with Pydantic and retries once on failure. If Gemini is unreachable, a template-based fallback report is generated from the scores so the app still works.

12. Frontend–Backend Integration Process

1. **Contract first:** the teammates publish FastAPI's OpenAPI doc at `http://localhost:8000/docs`; the frontend `types.ts` and `api.ts` mirror it.
2. **Mock mode:** until the backend is ready, `api.ts` uses a `VITE_USE_MOCKS=true` flag returning fake sessions, questions and a sample report, so Lovable's UI can be finished independently.
3. **CORS:** backend allows `http://localhost:8080` and `http://localhost:5173`.
4. **Dev proxy:** Vite proxies `/api` and `/ws` to port 8000, so the frontend uses relative URLs.
5. **Camera:** works on `localhost` without HTTPS (browsers treat localhost as secure).
6. **Integration test order:** health check → create session → fetch questions → submit answers → WebSocket frames → finish stage → generate report.
7. **Run together:** a root script starts `uvicorn` and `vite` at once (details in step 3 of this series).

13. Privacy and Ethics

- Explicit consent screen; guardian consent for under-18 users; camera can be stopped at any time.
- No raw video or images of the user are stored; only numeric features.
- All data is stored locally and a "delete all my data" button is always available.
- **Gemini is a cloud API.** Only the aggregated metrics and answers (no names, no images) are sent to it. State this clearly on the consent screen.
- API key lives only in `backend/.env`, never in the frontend.
- Language in the UI and report: "indicators" and "may suggest", never "has".

14. Milestones and Team Split

Phase	Frontend (Lovable)	Backend (teammates)
1	Landing, consent, profile, hub, mocked API	Project skeleton, DB, sessions API, health
2	Dyslexia and autism question UIs	Question banks, answers API, scoring for MCQs

Phase	Frontend (Lovable)	Backend (teammates)
3	Webcam hook, WebSocket sender, stimulus player	CV pipeline, WebSocket, aggregator
4	Camera check, calibration, ADHD and autism stages	ADHD and autism scoring
5	Report page and history	Gemini service, report builder, fallback
6	Polish, error states, accessibility	Threshold tuning, tests, cleanup

15. Risks and Limitations

- Laptop webcams give coarse gaze; expect region-level (left/right/top/bottom), not pixel-level, accuracy.
- Poor lighting or glasses glare reduce accuracy; the camera check step guards against this.
- Expression and repetitive-motion detection are heuristics and need tuning on real test data.
- Question content should be reviewed by a qualified professional and informed by established screening instruments before any real-world use.
- Thresholds are initial guesses and are not clinically validated.